Detección de Residuos Reciclables en una Cinta Transportadora con YOLOv8: una Comparación de Políticas de Aumentación

Julia Maldonado, Alejandro Valle, María Gabriela Di Grazia, Byron García
Carrera de Especialización en Inteligencia Artificial (CEIA)
Facultad de Ingeniería, Universidad de Buenos Aires (FIUBA)
Visión por Computadora II — Grupo 6

***Abstract*—Los centros de acopio reciben residuos mezclados cuya separación manual es lenta y costosa. Este trabajo propone un sistema de visión por computadora que, instalado sobre una cinta transportadora, detecta y clasifica residuos en seis categorías (biodegradable, cartón, vidrio, metal, papel y plástico) para priorizar la extracción de los materiales de mayor valor de reciclaje. Se entrenaron y compararon cuatro configuraciones – dos variantes de YOLOv8 (n y s) bajo dos políticas de aumentación de datos – sobre un conjunto de 10.464 imágenes (dataset Garbage Classification 3, Roboflow). La política de aumentación propuesta, diseñada a partir de un análisis cuantitativo de las carencias del dataset (orientación, iluminación y nitidez), obtuvo +1,2 puntos (YOLOv8s) y +0,7 puntos (YOLOv8n) de mAP@0.5:0.95 por encima de la configuración por defecto de Ultralytics en corridas de entrenamiento únicas, con YOLOv8s + proposed alcanzando 0,650 de mAP@0.5 y 55 FPS en una GPU T4.**

***Index Terms*—detección de objetos, YOLOv8, clasificación de residuos, aumentación de datos, visión por computadora, reciclaje.**

I. Introducción

En ciudades concurridas se genera una gran cantidad de basura mezclada. Separarla manualmente, material por material, es lento y costoso, y gran parte del material reciclable pierde valor comercial por no identificarse y extraerse a tiempo durante el procesamiento. Clasificar directamente en la vía pública, durante la recolección, es poco práctico por las condiciones de iluminación y movimiento; resulta más viable automatizar la clasificación una vez que el residuo llega a un centro de acopio o planta de procesamiento, donde puede pasar por una cinta transportadora bajo condiciones controladas de cámara.

El objetivo de este trabajo es diseñar y evaluar un sistema de detección de objetos basado en deep learning que, sobre una cinta transportadora, identifique en tiempo real la categoría de cada residuo y permita priorizar la extracción de los materiales de mayor valor de reciclaje (por ejemplo, aluminio, PET o cartón), en lugar de requerir una separación exhaustiva del 100% del flujo. El alcance de este trabajo se concentra en el problema de visión por computadora — detección y clasificación de los distintos tipos de residuos — dejando la implementación física del mecanismo de separación (brazo mecánico o aire a presión) como una extensión futura de la aplicación.

Como antecedente de un caso real, Koskinopoulou et al. [1] describen un sistema de categorización basado en visión por computadora para la separación robótica industrial de residuos reciclables, desplegado y evaluado en una planta de procesamiento de residuos real, bajo condiciones industriales exigentes, utilizando un módulo de visión de bajo costo basado en deep learning. En la industria, la prensa especializada reporta el despliegue de un robot de separación con IA de Recycleye en la planta de materiales recuperables (MRF) de re3 en Reading, Reino Unido, con 99% de pureza del material separado frente a 96% en la línea de separación humana de referencia y hasta 55 extracciones por minuto [2]. Ambos antecedentes respaldan la viabilidad de un enfoque de visión por computadora para esta tarea, aunque las cifras de la industria no están auditadas de forma independiente, a diferencia de las métricas reportadas en este trabajo.

II. Metodología

A. Conjunto de datos y procesamiento

Se utilizó el dataset público Garbage Classification 3 (v2, formato YOLOv8) de Roboflow [3], con 10.464 imágenes anotadas en seis clases: biodegradable, cardboard, glass, metal, paper y plastic. Previo al entrenamiento se auditó el export crudo para descartar imágenes corruptas y corregir o eliminar anotaciones defectuosas — un paso necesario porque los exports de Roboflow pueden contener errores de etiquetado que de otro modo perjudican el entrenamiento de forma silenciosa. Se re-dividió el dataset en una partición propia 70/20/10 (train/val/test) en lugar de conservar la de Roboflow, que resultaba desbalanceada entre clases y habría invalidado la evaluación por clase. El conjunto de test se mantuvo reservado y sin aumentar hasta la evaluación final.

B. Política de aumentación

Las técnicas de aumentación se eligieron a partir de un análisis cuantitativo del dataset procesado — no de supuestos genéricos — midiendo brillo, nitidez, color y balance de clases. El análisis mostró que al dataset le faltan imágenes oscuras o borrosas, condiciones esperables en video real de una cinta transportadora, y que el desbalance de clases observado se debe a la cantidad de objetos por foto y no

a la cantidad de fotos por clase. A partir de esos hallazgos se definieron dos configuraciones (*arms*) para comparar: `ultralytics_defaults`, la aumentación por defecto de Ultralytics [4] (mosaico, jitter HSV, flip horizontal, traslación y escala), y `proposed`, que conserva esos valores por defecto y añade flip vertical, oscurecimiento (reducción de brillo de hasta -40% con probabilidad 0,3), motion blur y Gaussian blur, para cubrir las carencias medidas de orientación, iluminación y nitidez, implementadas con Albumentations [5]. Cuando se le pasan transformaciones propias de Albumentations, Ultralytics reemplaza su conjunto integrado (Blur, MedianBlur, ToGray y CLAHE, cada una con probabilidad 0,01) en lugar de sumarlas, por lo que `proposed` también prescinde de esas cuatro transformaciones, que `ultralytics_defaults` conserva. La aumentación se aplica en forma *online* durante el entrenamiento, a partir de un único archivo de configuración compartido, de modo que todas las corridas usan exactamente la misma política y el conjunto de test nunca se aumenta por accidente.

C. Entrenamiento y evaluación

Se realizó fine-tuning de dos variantes pre-entrenadas de YOLOv8 [4] — YOLOv8n y YOLOv8s — bajo cada una de las dos políticas de aumentación, resultando en cuatro configuraciones entrenadas. Cada modelo se entrenó durante 50 épocas, batch size 16 y semilla fija (42), sobre una GPU T4 (Google Colab). La evaluación se realizó sobre el conjunto de test reservado, reportando mAP@0.5, mAP@0.5:0.95, F1 y latencia de inferencia extremo a extremo (200 imágenes de test, procesadas una por una), medida tanto en GPU T4 como en CPU de notebook (Intel i5-1135G7). El F1 es la media armónica de la precisión media y el recall medio que reporta Ultralytics.

III. Resultados Experimentales y Análisis

La Tabla I resume las métricas de las cuatro configuraciones sobre el conjunto de test. La política `proposed` obtuvo valores más altos que `ultralytics_defaults` en ambas arquitecturas: +1,2 puntos de mAP@0.5:0.95 en YOLOv8s y +0,7 puntos en YOLOv8n, con el mismo orden en el conjunto de validación (+1,4 y +1,1 puntos). Fue superior en cinco de las seis clases en ambos modelos, con una única excepción menor en CARDBOARD ($-0{,}008$). Con una sola corrida de entrenamiento por configuración, esto es una dirección consistente y no una diferencia sometida a un test estadístico. Tampoco puede atribuirse la ganancia a una transformación en particular: las dos recetas difieren en el flip vertical, el oscurecimiento, el desenfoque y las cuatro transformaciones integradas de Ultralytics que `proposed` deja de lado.

YOLOv8s superó a YOLOv8n en las seis clases bajo ambas recetas, a cambio de mayor latencia: en GPU T4, YOLOv8s + proposed procesó cada imagen en 18,1 ms (55 FPS) frente a 10,2 ms (98 FPS) de YOLOv8n + proposed. En CPU (Intel i5-1135G7, pesos `proposed`), YOLOv8n alcanzó 68,9 ms (14,5 FPS) frente a 177,3 ms (5,6 FPS) de YOLOv8s — una diferencia de 2,6× relevante para un despliegue sin GPU dedicada.

TABLE I
Métricas por modelo y receta (conjunto de test)

Modelo	Receta	mAP@0.5	mAP@.5:.95	F1
YOLOv8s	proposed	0.650	0.467	0.650
YOLOv8s	ultralytics_defaults	0.645	0.455	0.646
YOLOv8n	proposed	0.629	0.443	0.625
YOLOv8n	ultralytics_defaults	0.622	0.435	0.617

El modelo recomendado es YOLOv8s con la política `proposed`, por tener el mejor mAP@0.5:0.95 y F1 del conjunto y correr en tiempo real sobre GPU (55 FPS). Para un despliegue sin GPU se recomienda YOLOv8n + `proposed` como alternativa, con una caída de 2,5 puntos de mAP@0.5:0.95 a cambio de ser 2,6× más rápido en CPU. Las clases con peor desempeño en ambos modelos son BIODEGRADABLE, CARDBOARD y PAPER (mAP@0.5:0.95 entre 0,36 y 0,43 en YOLOv8s); la siguiente subsección analiza por qué.

A. Análisis de errores en el conjunto de test

La Fig. 1 muestra la matriz de confusión del modelo recomendado sobre el conjunto de test con confianza 0,25, el umbral que se usa en inferencia y en la demostración que sigue. La confusión entre materiales es poco frecuente: la mayor entrada fuera de la diagonal es METAL predicho como PLASTIC (5,5% de los objetos METAL), y ningún otro par supera el 3,3%. En particular, BIODEGRADABLE, CARDBOARD y PAPER casi nunca se confunden entre sí (como máximo 2,8%, PAPER predicho como CARDBOARD). El error dominante son los objetos no detectados: no se detecta el 51% de los objetos CARDBOARD, el 42% de BIODEGRADABLE y el 40% de PAPER, frente a 22–28% en GLASS, METAL y PLASTIC. Esto coincide con la brecha entre precisión (0,743) y recall (0,577) de la Tabla I. BIODEGRADABLE concentra además el 55% de las detecciones falsas sobre el fondo; es por lejos la clase con anotaciones más densas, con 4.504 de las 7.452 cajas de test (60%). Las clases débiles están limitadas, por lo tanto, por la detección y no por la confusión entre materiales.

B. Demostración sobre una cinta simulada

Para ilustrar el funcionamiento del sistema sin una cinta física se construyó una simulación: 11 imágenes del conjunto de test (elegidas al azar con semilla fija), cada una reescalada a un cuadrado de 420 px, se deslizan de izquierda a derecha sobre un fondo de banda de 1280×540 px, a 30 FPS durante 20 s (600 cuadros), como si pasaran frente a una cámara fija. El modelo recomendado (YOLOv8s + `proposed`, umbral de confianza 0,25) se ejecuta cuadro por cuadro, igual que lo haría un sistema en línea. La Fig. 2 muestra seis cuadros representativos.

La latencia por cuadro (preprocesado, inferencia y NMS), medida tras 10 cuadros de calentamiento no cronometrados

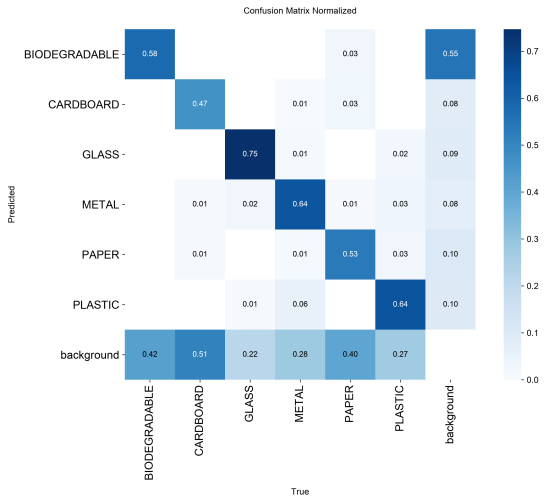


Fig. 1. Matriz de confusión normalizada de YOLOv8s + `proposed` sobre el conjunto de test con confianza 0,25 (columnas: clase real). La última fila es la fracción de objetos no detectados; la última columna distribuye las detecciones falsas sobre el fondo.

sobre la GPU integrada (backend MPS) de un Apple M5 Pro, fue de 6,35 ms en promedio (157 FPS), con una mediana de 6,45 ms y un percentil 95 de 7,09 ms, muy por encima de los 30 FPS del video simulado.

TABLE II
DETECCIONES POR CLASE EN LA DEMOSTRACIÓN (CONTEO POR CUADRO)

Clase	**Detecciones**	**Confianza media**
GLASS	1241	0.616
METAL	746	0.617
PLASTIC	482	0.487
PAPER	188	0.686
BIODEGRADABLE	53	0.554
CARDBOARD	5	0.396

La Tabla II cuenta detecciones *por cuadro*: un mismo objeto aparece en muchos cuadros consecutivos, de modo que la tabla refleja las imágenes que recorrieron la banda y no la cantidad de objetos distintos (para ello haría falta seguimiento, ver Conclusiones). En los cuadros inspeccionados se observaron dos comportamientos. Por un lado, los objetos de apariencia clara se clasifican con confianza alta, por ejemplo las botellas de vidrio erguidas (0,82 a 0,86). Por otro, aparecen confusiones entre clases: un mismo bidón plástico blanco se etiquetó como PLASTIC (0,61) en un cuadro y como METAL (0,66) en otro; un vaso de papel se etiquetó como PAPER (0,76) y, al entrar por el borde de la banda parcialmente recortado, como METAL (0,61); un frasco de vidrio se etiquetó como METAL; y los restos de aguacate en un recipiente se etiquetaron como GLASS en lugar de BIODEGRADABLE. La clase se vuelve inestable cuando el objeto aparece cortado en el borde del encuadre. CARDBOARD casi no se detectó (5 detecciones, confianza media 0,396), coherente con su bajo desempeño en test. Estas observaciones son cualitativas y se basan en 11 imágenes distintas, por lo que no sustituyen la evaluación cuantitativa de la Tabla I.

Limitaciones: cada modelo y receta se entrenó una única vez, por lo que las diferencias reportadas no fueron sometidas a un test de significancia estadística. Las dos recetas difieren en varias transformaciones a la vez, por lo que la comparación no aísla el efecto de ninguna en particular, y se entrenaron con versiones consecutivas de Ultralytics (8.4.173 para `ultralytics_defaults` y 8.4.172 para `proposed`). De las cuatro recetas definidas en la política de aumentación, solo dos (`ultralytics_defaults` y `proposed`) fueron efectivamente entrenadas; `no_augmentation` y `proposed_low_saturation` quedan como trabajo futuro. Además, el dataset utilizado, aunque realista, no replica completamente el nivel de desorden visual (objetos superpuestos, oclusiones parciales) que literatura especializada documenta en video real de cintas transportadoras [6]; validar el modelo recomendado contra imágenes propias de la banda, más cercanas a ese escenario, queda como un paso natural antes de un despliegue productivo.

IV. CONCLUSIONES

1) Una política de aumentación diseñada a partir de carencias medidas cuantitativamente en el dataset (orientación, iluminación, nitidez) obtuvo un mAP@0.5:0.95 más alto que los valores por defecto de Ultralytics en ambas arquitecturas y tanto en validación como en test, sin necesidad de recolectar más datos. Con una corrida por configuración y recetas que difieren en varias transformaciones, confirmar el efecto y atribuirlo a transformaciones específicas requiere corridas repetidas y una ablación.
2) YOLOv8s ofrece mejor precisión que YOLOv8n en las seis clases bajo ambas recetas, pero a un costo de latencia relevante en CPU (2,6× más lento que YOLOv8n), por lo que la elección entre ambos depende directamente del hardware de despliegue disponible en el centro de acopio.
3) Re-dividir el dataset en lugar de usar la partición original de Roboflow fue necesario para obtener una evaluación por clase confiable, dado el desbalance detectado en la partición original.
4) Las clases BIODEGRADABLE, CARDBOARD y PAPER concentran el menor desempeño del sistema, y su error principal en test son los objetos no detectados (40–51% sin detectar con confianza 0,25), no la confusión con otros materiales (como máximo 2,8%). Mejorar el recall de estas clases, por ejemplo en las imágenes de BIODEGRADABLE con anotaciones densas, es el cuello de botella principal a resolver en una futura iteración.
5) La demostración sobre una cinta simulada corrió a 157 FPS en una GPU integrada (Apple M5 Pro, MPS), muy por encima de los 30 FPS del video, pero mostró confusiones, sobre todo METAL frente a PLASTIC, PAPER y GLASS, y clases inestables cuando el objeto

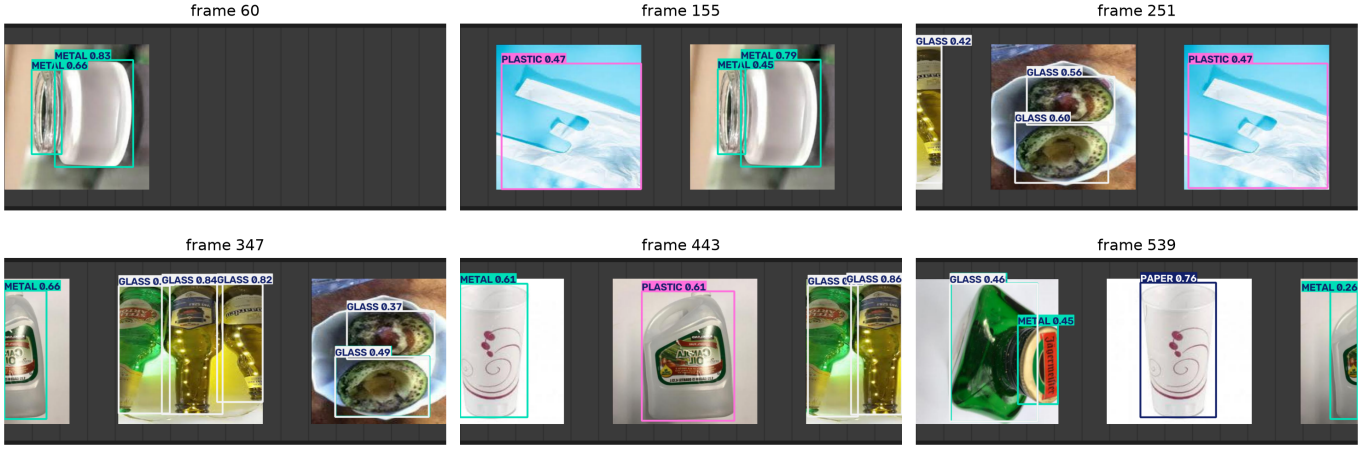


Fig. 2. Seis cuadros de la demostración sobre la cinta simulada, con las detecciones del modelo YOLOv8s + `proposed`. Se observan aciertos (botellas de vidrio, vaso de papel) y confusiones entre clases, en particular METAL.

queda cortado en el borde del encuadre. Esto refuerza que el siguiente paso es validar con imágenes propias de la banda y con objetos parcialmente visibles.

6) Como trabajo futuro, la arquitectura puede extenderse incorporando seguimiento de objetos (*tracking*) para evitar contar el mismo residuo dos veces en video, y evaluando directamente la implementación física del mecanismo de separación, que quedó fuera del alcance de este trabajo.

REFERENCES

[1] M. Koskinopoulou, F. Raptopoulos, G. Papadopoulos, N. Mavrakis, and M. Maniadakis, "Robotic waste sorting technology: Toward a vision-based categorization system for the industrial robotic separation of recyclable waste," *IEEE Robotics & Automation Magazine*, vol. 28, no. 2, pp. 50–60, 2021.

[2] letsrecycle.com, "re3 achieves 99% purity with AI picking robot." https://www.letsrecycle.com/news/re3-achieves-99-purity-with-ai-picking-robot/, 2022. Accessed: 2026-10-08.

[3] Material Identification, "Garbage classification 3." https://universe.roboflow.com/material-identification/garbage-classification-3, 2024. Roboflow Universe. Accessed: 2026-10-08.

[4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8." https://github.com/ultralytics/ultralytics, 2023.

[5] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," *Information*, vol. 11, no. 2, p. 125, 2020.

[6] D. Bashkirova, M. Abdelfattah, Z. Zhu, J. Akl, F. Alladkani, P. Hu, V. Ablavsky, B. Calli, S. A. Bargal, and K. Saenko, "Zerowaste dataset: Towards deformable object segmentation in cluttered scenes," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.